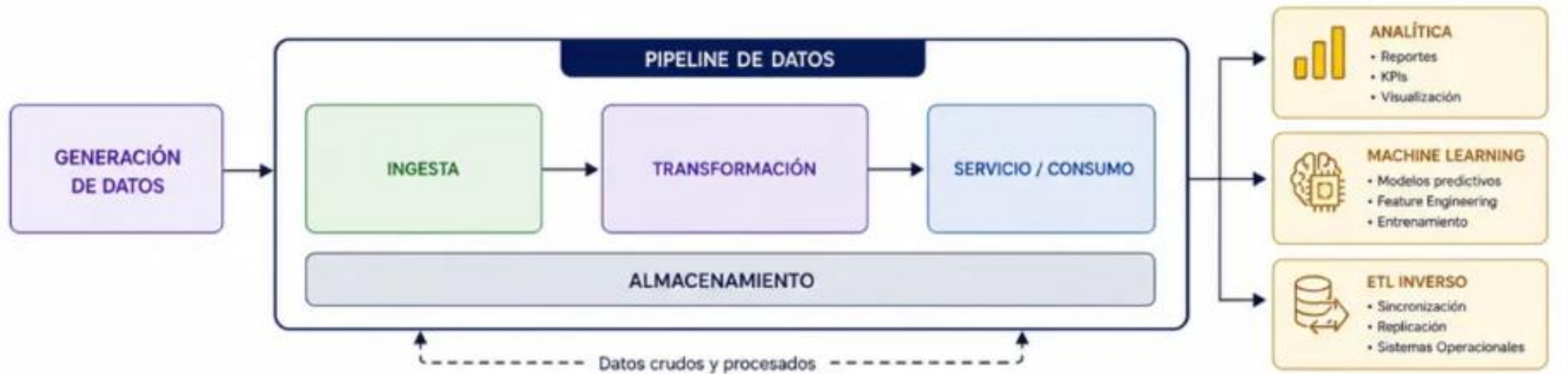


AWS Lambda en un data pipeline

Lecciones aprendidas para armar tu solución

Analítica = Data pipeline



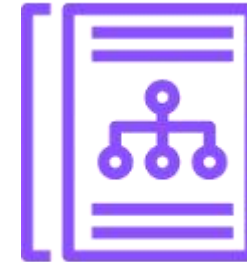
¡AWS Glue al rescate!



**Varios tipos
de Job**



**AWS Glue Catalog
como integración**



Workflows

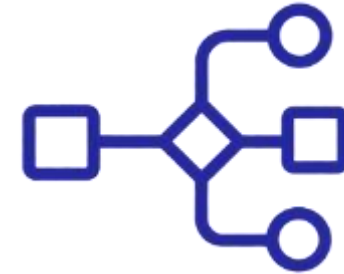
No todo es AWS Glue en la vida



Volumen de
datos



Velocidad de
datos

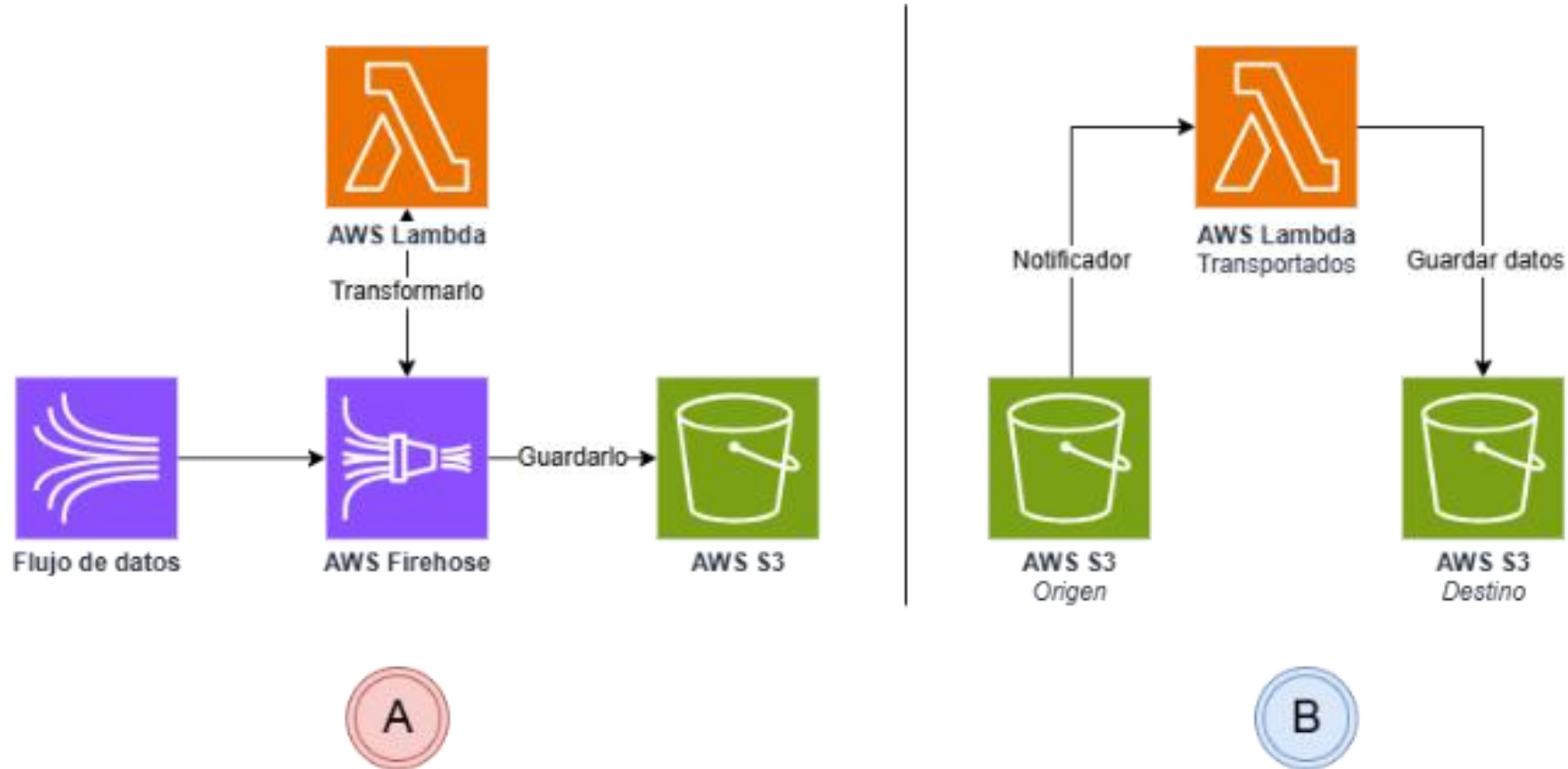


Complejidad
de la solución

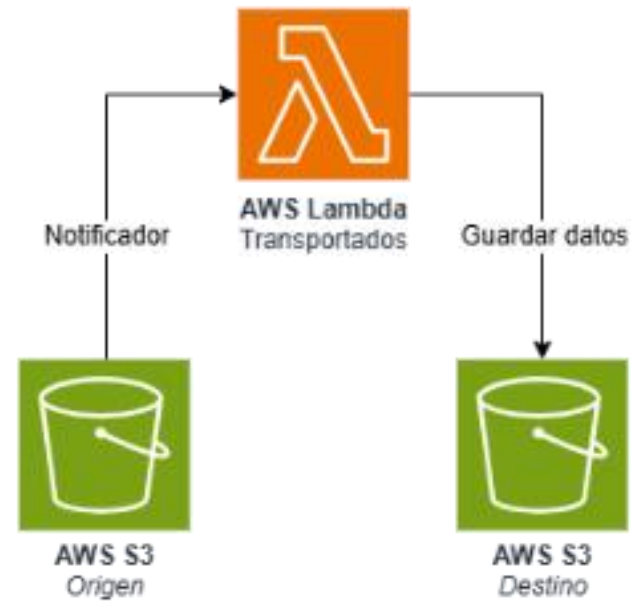
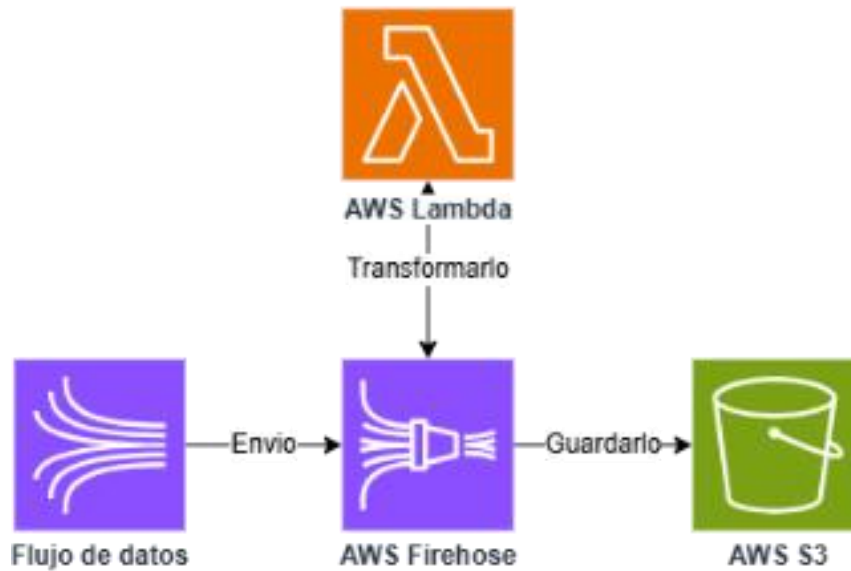
¿AWS Lambda como respuesta?



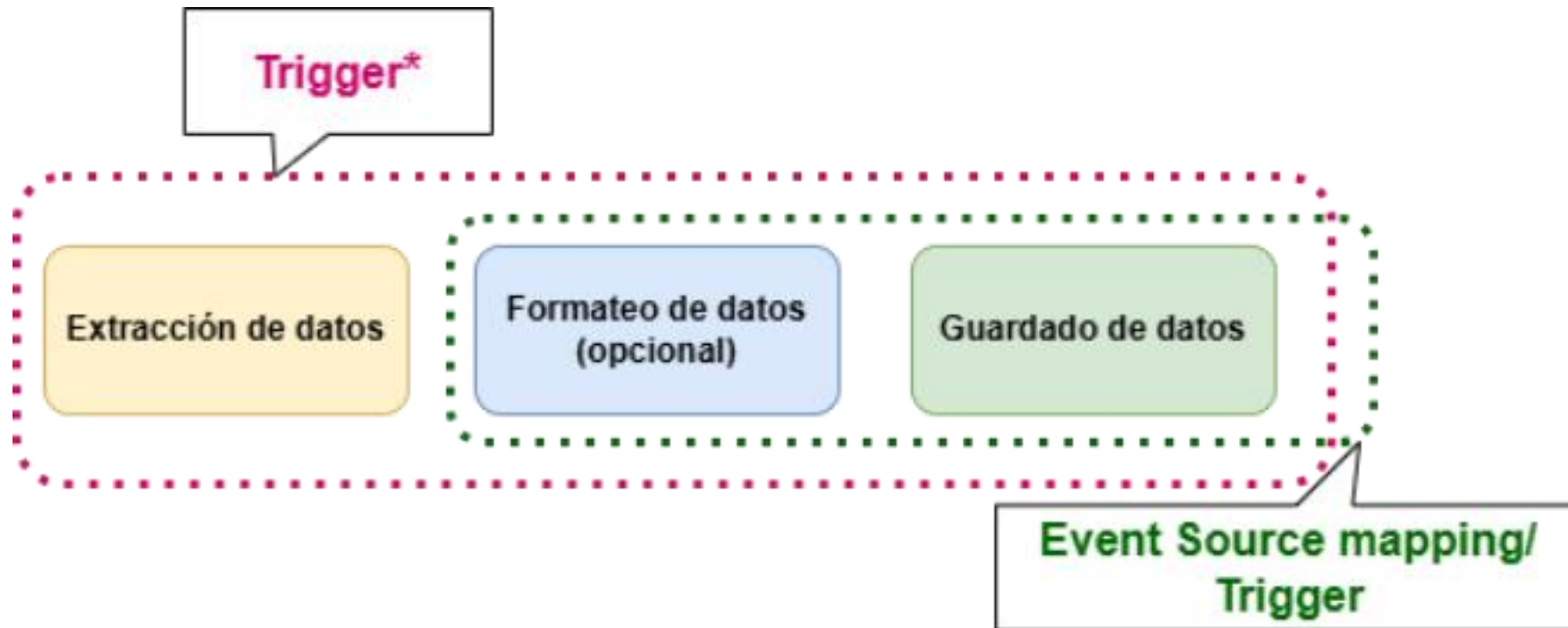
AWS Lambda - Ingesta: Límites



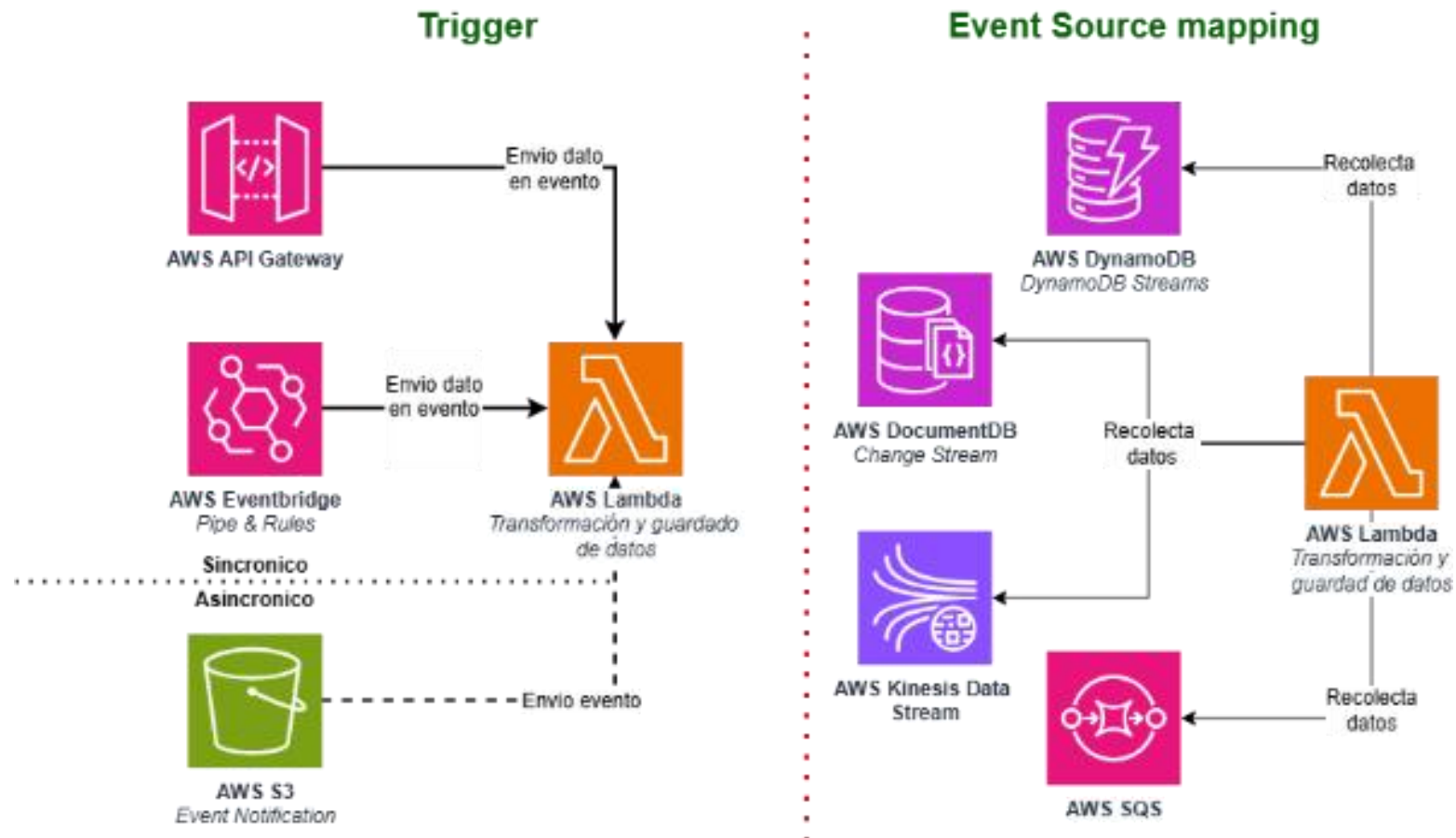
AWS Lambda - Ingesta: Límites



AWS Lambda - Ingesta: Definición

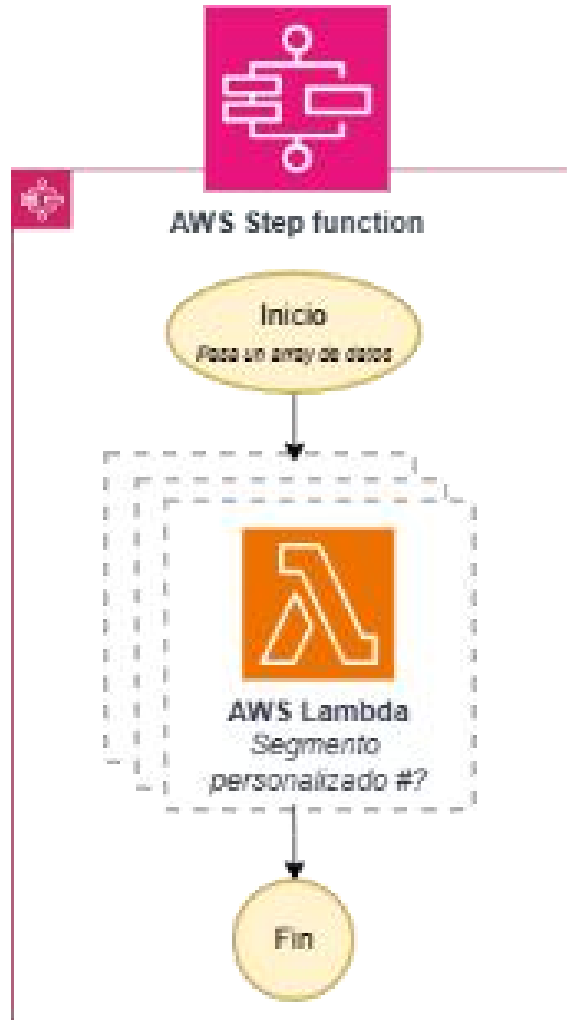


AWS Lambda - Ingesta: Método #1



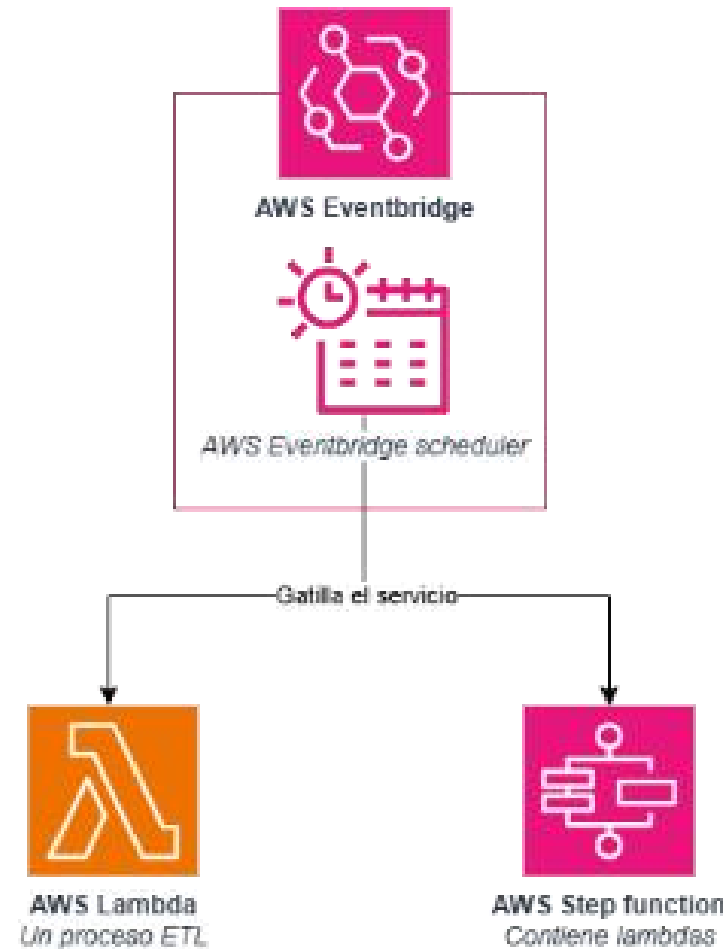
AWS Lambda - Ingesta: Método #2

```
{
  "executionID": "process-001",
  "files": [
    {
      "fileName": "clientes_01.parquet",
      "bucket": "mi-data-lake",
      "key": "raw/clientes/clientes_01.parquet"
    },
    {
      "fileName": "clientes_02.parquet",
      "bucket": "mi-data-lake",
      "key": "raw/clientes/clientes_02.parquet"
    },
    {
      "fileName": "clientes_03.parquet",
      "bucket": "mi-data-lake",
      "key": "raw/clientes/clientes_03.parquet"
    },
    {
      "fileName": "clientes_04.parquet",
      "bucket": "mi-data-lake",
      "key": "raw/clientes/clientes_04.parquet"
    },
    {
      "fileName": "clientes_05.parquet",
      "bucket": "mi-data-lake",
      "key": "raw/clientes/clientes_05.parquet"
    }
  ]
}
```



```
{
  "Comment": "Ejemplo de procesamiento paralelo con Lambda",
  "StartAt": "ProcessFiles",
  "States": {
    "ProcessFiles": {
      "Type": "Map",
      "ItemsPath": "$.files",
      "MaxConcurrency": 3,
      "Iterator": {
        "StartAt": "ProcessFile",
        "States": {
          "ProcessFile": {
            "Type": "Task",
            "Resource": "arn:aws:states:::lambda:invoke",
            "Parameters": {
              "FunctionName": "process-file-lambda",
              "Payload.$": "$"
            },
            "End": true
          }
        }
      }
    }
  }
}
```

AWS Lambda - Ingesta: Soporte



AWS Lambda - Ingesta: Lecciones



Lección #1

**Entiende cómo es
el comportamiento
de los datos**



Lección #2

**Perfila el volumen
de los datos:
costos y configuración**



Lección #3

**Apoyate en servicios AWS
para crear o viabilizar
soluciones**

AWS Lambda - Transformación: Base



ClickHouse



DuckDB

Propiedades técnicas interesantes:

1. In-Process / embedded
2. Orientado a OLAP
3. Lazy evaluation
4. Procesamiento con threads
5. Vectorized query execution model
6. Gestión de memoria y Spill-to-disk
7. Formatos populares: CSV, JSON y Parquet

AWS Lambda - Transformación: Instalación

	ZIP File Archive	AWS Lambda Layer	Container Image
 ClickHouse			
 DuckDB			

AWS Lambda - Transformación: patrón

Q1: SELECT * FROM read_parquet(<Table>)

Q2: SELECT id_tienda, tipo_producto, SUM (costo_compra) FROM read_parquet(<Table>)
GROUP BY id_tienda, tipo_producto

Q3: SELECT id_tienda, tipo_producto, producto_comprado, SUM (costo_compra) FROM
read_parquet(<Table>) GROUP BY id_tienda, tipo_producto, producto_comprado

Q4: SELECT id_compra, count (*) as cantidad FROM read_parquet(<Table>) GROUP BY
id_compra (columna "id_compra" con altamente cardinal)

AWS Lambda - Transformación: patrón



	Q1 - 100MB	Q1 - 300MB	Q1 - 500MB	Q1 - 800MB	Q1 - 1GB
ChDB	OOO	OOO	OOO	OOO	OOO
DuckDB	33.81149 seg	OOO	OOO	OOO	OOO

	Q2 - 100MB	Q2 - 300MB	Q2 - 500MB	Q2 - 800MB	Q2 - 1GB
ChDB	5.6388 seg	10.63723 seg	17.11011 seg	25.23828	30.1376 seg
DuckDB	2.14177 seg	5.24080 seg	9.49515 seg	12.2754 seg	13.397670 seg

	Q3 - 100MB	Q3 - 300MB	Q3 - 500MB	Q3 - 800MB	Q3 - 1GB
ChDB	6.06761 seg	13.297 seg	21.1739 seg	31.2138 seg	37.2896 seg
DuckDB	2.4854 seg	5.71321 seg	10.5305 seg	12.48280 seg	14.7741 seg

	Q4 - 100MB	Q4 - 300MB	Q4 - 500MB	Q4 - 800MB	Q4 - 1GB
ChDB	36.9501 seg	OOO	OOO	OOO	OOO
DuckDB	OOO	OOO	OOO	OOO	OOO



DuckDB

AWS Lambda - Transformación: Código



ClickHouse



DuckDB

```
import os
import time

import clickhouse_connect

### PARAMETROS GENERALES
S3_BUCKET_SOURCE = os.environ["S3_BUCKET_SOURCE"]
S3_PREFIX_SOURCE = os.environ["S3_PREFIX_SOURCE"]
S3_PATH_TARGET = os.environ["S3_PATH_TARGET"]
SQL_QUERY_TYPE = os.environ["SQL_QUERY_TYPE"]
FILE_NUMBER = os.environ["FILE_NUMBER"]

print("S3_BUCKET_SOURCE: ", S3_BUCKET_SOURCE)
print("S3_PREFIX_SOURCE: ", S3_PREFIX_SOURCE)
print("S3_PATH_TARGET: ", S3_PATH_TARGET)
print("SQL_QUERY_TYPE: ", SQL_QUERY_TYPE)

query_sql_1 = """
INSERT INTO FUNCTION s3(
    's3://temporaltable-data/datos_base/results_clickdb/(S3_PATH_TARGET)/part_{:partition_id}.parquet',
    'parquet'
)
PARTITION BY cityhash64(id_cliente) % (FILE_NUMBER)
SELECT id_cliente, tipo_producto, producto_comprado, SUM(costo_compra)
FROM s3(
    's3://{S3_BUCKET_SOURCE}/{S3_PREFIX_SOURCE}/'.parquet'
)
GROUP BY id_cliente, tipo_producto, producto_comprado
"""

def lambda_handler(event, context):
    query_sql = query_sql_1

    print("La consulta seleccionada fue: ", query_sql)
    start = time.time()
    clickhouse_connect.query(query_sql)
    end_setup = time.time()
    print("Elapsed time (Loading data): ", end_setup - start, "seconds")
```

```
import time
import os
import duckdb

os.makedirs("/tmp/clickdb_tmp", exist_ok=True)
os.makedirs("/tmp/home_tmp", exist_ok=True)

def lambda_handler(event, context):
    start = time.time()
    con = duckdb.connect()
    con.execute("SET home_directory = '/tmp/home_tmp';")
    con.execute("SET temp_directory = '/tmp/clickdb_tmp';")
    con.execute("SET memory_limit = '200MB';")

    # cargar extensiones necesarias
    con.execute("INSTALL httpfs;")
    con.execute("LOAD httpfs;")
    con.execute("INSTALL parquet;")
    con.execute("LOAD parquet;")

    con.execute("SET threads = 1")
    con.execute("SET preserve_insertion_order = false")
    end_setup = time.time()
    print("Elapsed time (setup): ", end_setup - start, "seconds")

    query = """
    COPY (
        SELECT id_compra, count(*) as counting
        FROM read_parquet('s3://dataset--test--structured-data/parquet_files/100MB/*.parquet')
        GROUP BY id_compra
    )
    TO 's3://temporaltable-data/datos_base/results_clickdb/100MB_movingst'
    (
        FORMAT PARQUET,
        PER_THREAD_OUTPUT true,
        FILE_SIZE_BYTES '10MB'
    );
    """

    result = con.execute(query)
```

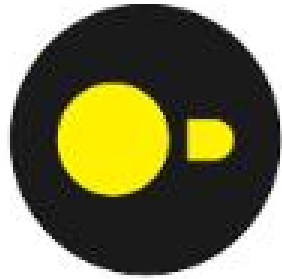
AWS Lambda - Transformación: mejora



AWS Lambda - Transformación: Revisión

			
 ClickHouse	Read Write: Append	Read Write: Append	Read
 DuckDB	Read Write: Append	Read Write: Append Write: Merge	

AWS Lambda - Transformación: ¿Cuál utilizar?



DuckDB

“El Maduro”



ClickHouse

“El Eficiente”

AWS Lambda - Transformación: Lecciones



Para procesos con
enfoque incremental
o de sobreescritura.



Es más para
transformación
exclusivamente.



Se estratégico en
como construir
las transformaciones.

***"La elección del servicio correcto
siempre va a depender
del entendimiento del requerimiento
para diseñar la solución"***

¿Quién Soy?

Roles

- Data Engineer en TRD Retail Corp.
- Data mentor
- AWS Community Builder - Data
- AWS User Group Leader - AWS UG Arequipa

AWS Builder Center



Linkedin



¿Preguntas o Dudas?



AWS Lambda en un data pipeline

Lecciones aprendidas para armar tu solución